Single-Integrator to Unicycle Velocity Conversion

Mathematical Derivation and Implementation Notes

Multi-Robot Formation Control — ArUco Real-World Edition

June 26, 2026

Contents

1 Overview **1**

2 Unicycle Kinematic Model **2**

3 Look-Ahead Point Construction **2**

4 Inversion: Unicycle Commands from Single-Integrator Velocity **2**
4.1 Invertibility 2
4.2 Closed-Form Inverse 3

5 Implementation in Code **3**
5.1 Linear velocity gain 3
5.2 Angular velocity 3
5.3 Compact matrix form 3
5.4 Velocity saturation 3

6 Signal Flow Diagram **3**

7 Parameter Summary **3**

8 Geometric Interpretation **3**

9 Complete Conversion — Step by Step **4**

1 Overview

The Double-Angle Visual Cone CBF controller produces *single-integrator* velocity commands

$$\dot{p}_i = \begin{bmatrix} \dot{p}_{x,i} \\ \dot{p}_{y,i} \end{bmatrix} \in \mathbb{R}^2$$

for each robot i. Physical TurtleBot3 robots, however, are *unicycle* systems commanded via linear speed v_i and angular speed ω_i. This document derives the transformation that converts $\dot{p}_i$ into (v_i, ω_i) and documents the implementation choices made in `multi_unicycle_controller.py`.

2 Unicycle Kinematic Model

A unicycle robot at configuration $(x_i,\, y_i,\, \theta_i)$ obeys

$$\begin{bmatrix} \dot{x}_i \\ \dot{y}_i \\ \dot{\theta}_i \end{bmatrix} = \begin{bmatrix} \cos\theta_i & 0 \\ \sin\theta_i & 0 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} v_i \\ \omega_i \end{bmatrix}, \tag{1}$$

where $v_i \in \mathbb{R}$ is the *linear* (forward) speed and $\omega_i \in \mathbb{R}$ is the *angular* speed.

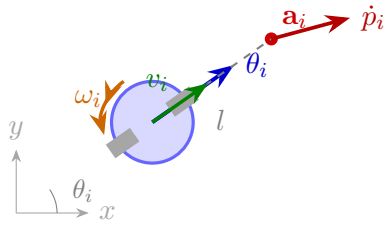


Figure 1: Unicycle robot with centre (x_i, y_i), heading θ_i, linear speed v_i, angular speed ω_i, and look-ahead point $\mathbf{a}_i$ at distance l along the heading axis. The single-integrator velocity $\dot{p}_i$ is prescribed at $\mathbf{a}_i$.

3 Look-Ahead Point Construction

To make the unicycle system amenable to single-integrator design, we introduce a *look-ahead* (auxiliary) point $\mathbf{a}_i$ located a distance $l > 0$ ahead of the robot along its heading:

$$\mathbf{a}_i = \begin{bmatrix} x_i + l\cos\theta_i \\ y_i + l\sin\theta_i \end{bmatrix}. \tag{2}$$

Differentiating with respect to time and substituting (1):

$$\dot{a}_{x,i} = \dot{x}_i - l\sin\theta_i\,\dot{\theta}_i = v_i\cos\theta_i - l\sin\theta_i\,\omega_i, \tag{3}$$

$$\dot{a}_{y,i} = \dot{y}_i + l\cos\theta_i\,\dot{\theta}_i = v_i\sin\theta_i + l\cos\theta_i\,\omega_i. \tag{4}$$

In matrix form:

$$\underbrace{\begin{bmatrix} \dot{a}_{x,i} \\ \dot{a}_{y,i} \end{bmatrix}}_{\dot{p}_i} = \underbrace{\begin{bmatrix} \cos\theta_i & -l\sin\theta_i \\ \sin\theta_i & l\cos\theta_i \end{bmatrix}}_{M(\theta_i,l)} \begin{bmatrix} v_i \\ \omega_i \end{bmatrix}. \tag{5}$$

4 Inversion: Unicycle Commands from Single-Integrator Velocity

4.1 Invertibility

The determinant of $M(\theta_i, l)$ is

$$\det M = l\cos^2\theta_i + l\sin^2\theta_i = l > 0 \quad \forall\,\theta_i,\ l \neq 0, \tag{6}$$

so M is always invertible for any non-zero look-ahead distance.

4.2 Closed-Form Inverse

Using the formula for a 2×2 inverse:

$$M^{-1}(\theta_i, l) = \frac{1}{l} \begin{bmatrix} l\cos\theta_i & l\sin\theta_i \\ -\sin\theta_i & \cos\theta_i \end{bmatrix} = \begin{bmatrix} \cos\theta_i & \sin\theta_i \\ -\dfrac{\sin\theta_i}{l} & \dfrac{\cos\theta_i}{l} \end{bmatrix}. \tag{7}$$

Therefore the unicycle commands that realise any desired single-integrator velocity $\dot{p}_i$ are

$$\boxed{\begin{bmatrix} v_i \\ \omega_i \end{bmatrix} = M^{-1}(\theta_i, l)\, \dot{p}_i = \begin{bmatrix} \cos\theta_i\, \dot{p}_{x,i} + \sin\theta_i\, \dot{p}_{y,i} \\ \dfrac{-\sin\theta_i\, \dot{p}_{x,i} + \cos\theta_i\, \dot{p}_{y,i}}{l} \end{bmatrix}.} \tag{8}$$

5 Implementation in Code

5.1 Linear velocity gain

The implementation uses a gain of 2 on v_i:

$$v_i = 2\big(\dot{p}_{x,i}\cos\theta_i + \dot{p}_{y,i}\sin\theta_i\big). \tag{9}$$

This effectively scales the CBF output by a factor of 2 in the forward-speed direction, increasing responsiveness without affecting angular convergence (the ω_i formula is unchanged from (8)).

5.2 Angular velocity

$$\omega_i = \frac{-\dot{p}_{x,i}\sin\theta_i + \dot{p}_{y,i}\cos\theta_i}{l}, \qquad l = 0.1 \text{ m}. \tag{10}$$

5.3 Compact matrix form

Combining (9) and (10):

$$\begin{bmatrix} v_i \\ \omega_i \end{bmatrix} = \underbrace{\begin{bmatrix} 2\cos\theta_i & 2\sin\theta_i \\ -\dfrac{\sin\theta_i}{l} & \dfrac{\cos\theta_i}{l} \end{bmatrix}}_{T(\theta_i)} \begin{bmatrix} \dot{p}_{x,i} \\ \dot{p}_{y,i} \end{bmatrix}. \tag{11}$$

5.4 Velocity saturation

To respect physical actuator limits, both commands are clipped:

$$v_i \leftarrow \text{clip}(v_i,\, -v_{\max},\, +v_{\max}), \qquad v_{\max} = 0.18 \text{ m/s}, \tag{12}$$

$$\omega_i \leftarrow \text{clip}(\omega_i,\, -\omega_{\max},\, +\omega_{\max}), \qquad \omega_{\max} = 1.50 \text{ rad/s}. \tag{13}$$

6 Signal Flow Diagram

7 Parameter Summary

8 Geometric Interpretation

The transformation $T(\theta_i)$ decomposes the desired single-integrator velocity $\dot{p}_i$ into:

1. **Forward component** (parallel to heading): $\dot{p}_{x,i}\cos\theta_i + \dot{p}_{y,i}\sin\theta_i \longrightarrow$ proportional to v_i.
2. **Lateral component** (perpendicular to heading): $-\dot{p}_{x,i}\sin\theta_i + \dot{p}_{y,i}\cos\theta_i \longrightarrow$ proportional to ω_i (scaled by $1/l$).

Intuitively, a large lateral error steers the robot; a large forward error accelerates it.

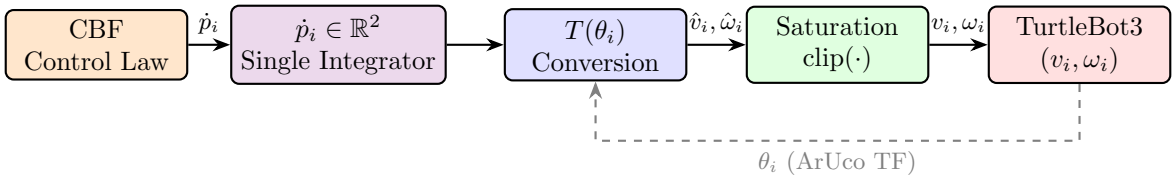


Figure 2: Signal flow for robot i. The CBF law outputs a single-integrator velocity $\dot{p}_i$; the look-ahead transformation $T(\theta_i)$ converts it to unicycle commands; saturation enforces actuator limits; the robot executes (v_i, ω_i). The heading θ_i is fed back from the ArUco TF2 measurement.

Table 1: Parameters used in the unicycle conversion.

Symbol	**Variable in code**	**Value**	**Description**
l	`self.length`	0.10 m	Look-ahead distance
$v_{\max}$	`self.v_max`	0.18 m/s	Max linear speed
$\omega_{\max}$	`self.omega_max`	1.50 rad/s	Max angular speed
v-gain	(hardcoded)	2	Extra linear gain

9 Complete Conversion — Step by Step

Given the CBF output $\dot{p}_i = [\dot{p}_{x,i},\, \dot{p}_{y,i}]^\top$ and the current ArUco heading θ_i:

1. **Compute forward speed**

$$\hat{v}_i = 2\big(\dot{p}_{x,i}\cos\theta_i + \dot{p}_{y,i}\sin\theta_i\big).$$

2. **Compute angular speed**

$$\hat{\omega}_i = \frac{-\dot{p}_{x,i}\sin\theta_i + \dot{p}_{y,i}\cos\theta_i}{l}.$$

3. **Saturate**

$$v_i = \text{clip}(\hat{v}_i,\, -v_{\max},\, +v_{\max}), \qquad \omega_i = \text{clip}(\hat{\omega}_i,\, -\omega_{\max},\, +\omega_{\max}).$$

4. **Publish** $(v_i,\, \omega_i)$ as a `TwistStamped` message on `/bot_`i`/cmd_vel`.

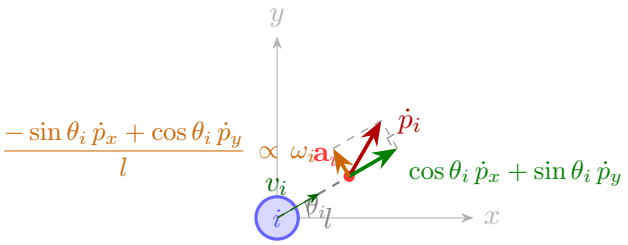


Figure 3: Geometric decomposition of $\dot{p}_i$ at the look-ahead point $\mathbf{a}_i$ (heading $\theta_i = 30°$, example). The green component (parallel to heading) drives forward speed v_i. The orange component (perpendicular to heading), divided by l, drives angular speed ω_i.